

GPUMODE Lecture Study Notes: Lecture 85:

Factorio Learning Environment

Core Problem the Lecture Addresses

The lecture presents the **Factorio Learning Environment**, abbreviated here as **FLE**, as an evaluation and research platform for long-horizon language-model agents. The core claim is that most existing agent benchmarks saturate too quickly: as models improve, a benchmark that once differentiated agents becomes too easy, and the evaluation stops providing meaningful signal.

The central problem is therefore:

How can we evaluate and study agents in an environment that is open-ended, procedurally variable, long-horizon, consequential, resistant to memorization, and hard enough to remain useful as frontier models improve?

Factorio is proposed as a strong candidate because it is not merely a video game task. It is a spatial, embodied, resource-constrained, systems-engineering environment in which the agent must build increasingly complex production systems over many thousands of actions. The environment requires planning, debugging, abstraction, spatial reasoning, resource allocation, incremental engineering, error correction, and long-term compounding improvement.

The lecture frames the motivation around several empirical trends:

- **Benchmark saturation:** static agent benchmarks lose usefulness once models reach high success rates.
- **Increasing task horizon:** the length of tasks that frontier agents can complete appears to be increasing rapidly.
- **Need for ultra-long-horizon environments:** future agents may need to operate over hundreds of hours and tens of thousands of tool calls.
- **Need for open-ended performance ceilings:** a benchmark should not end at a single binary completion threshold.
- **Need to separate knowledge from intelligence:** procedurally generated environments reduce the chance that agents simply memorize solutions.

Factorio provides a natural setting for this because the player starts in a generated world, mines resources, automates production, researches technologies, builds infrastructure, and eventually launches a rocket. However, launching a rocket is not the true ceiling. Human players can continue scaling to mega-bases that launch many rockets per minute, and Factorio also contains unbounded research loops whose costs grow over time.

Required Background

Factorio as a Systems-Engineering Environment

In Factorio, an agent controls an embodied character in a two-dimensional world. The character has position, reach, inventory, and local interaction constraints. The agent must:

- mine raw resources such as coal, stone, iron ore, and copper ore;
- place machines such as burner mining drills, electric mining drills, furnaces, assemblers, belts, inserters, pipes, pumps, and power infrastructure;
- route items between machines using belts, inserters, chests, pipes, trains, or other logistics mechanisms;
- craft intermediate items such as plates, gears, circuits, steel, plastic, sulfur, advanced circuits, batteries, and low-density structures;
- invest in research to unlock new technologies;
- defend against enemies if pollution attracts attacks;
- scale production throughput while maintaining correctness.

A useful mental analogy is that Factorio turns software architecture into physical geometry. A software program has modules, interfaces, control flow, data dependencies, resource bottlenecks, and bugs. A Factorio factory has machines, belts, recipes, item flows, resource dependencies, spatial bottlenecks, and broken layouts.

Why Factorio Is Hard for Agents

Factorio is difficult because small errors have persistent consequences. A single misplaced inserter, a belt facing the wrong direction, or an assembler with the wrong recipe can break an entire production chain. Unlike a short question-answering task, a failed action leaves state behind. The next action must reason over the corrupted world.

The difficulty is amplified by several properties:

- **Embodiment:** the agent cannot directly edit the whole world; it must move and act through a character.

- **Spatial layout:** production requires correct two-dimensional placement and routing.
- **Long dependency chains:** advanced items require many intermediate ingredients.
- **Delayed reward:** early infrastructure decisions pay off only many steps later.
- **Compounding decisions:** many slightly good decisions compound into a strong factory.
- **State persistence:** mistakes remain in the world unless repaired.
- **Resource trade-offs:** speed, pollution, military defense, clean energy, and scalability interact.
- **Open-ended scaling:** there is no single maximum score.

Agent Evaluation Terminology

The lecture uses several agent-evaluation concepts:

- **Policy:** the code or action sequence produced by the model at a given step.
- **Tool call:** a primitive interaction with the environment, such as moving, harvesting, placing, picking up, or observing.
- **Trajectory:** the full sequence of observations, model actions, tool executions, errors, and rewards over an episode.
- **Lab play:** a constrained scenario where resources are available nearby and the agent focuses on factory construction.
- **Open play:** a full-world scenario closer to a normal Factorio run.
- **Production score:** a reward-like measure based on net item production value.
- **Milestones:** additional indicators of exploration and progression through the item or technology space.

Why This Lecture Belongs in a Systems Curriculum

Although the lecture is not about CUDA kernels, it is deeply relevant to GPU systems and deep learning systems research because it asks how to evaluate agents that write code, manage state, debug long-running systems, and optimize throughput. The same themes appear in high-performance computing:

- long-horizon performance optimization;
- resource scheduling;

- throughput measurement;
- bottleneck diagnosis;
- hierarchical abstraction;
- stateful debugging;
- distributed rollout execution;
- simulation throughput versus model-inference throughput.

A Factorio agent is not optimizing shared-memory tiles or tensor cores directly, but it is solving a systems problem: build a production pipeline that transforms low-value inputs into high-value outputs under constraints.

Main Concepts

Why Existing Benchmarks Saturate

The lecture argues that many agent benchmarks follow a lifecycle:

1. A benchmark is introduced and appears difficult.
2. Models improve through scaling, post-training, tool use, and better prompting.
3. Success rates rise until the benchmark no longer differentiates agents.
4. The benchmark becomes less informative for frontier evaluation.

This creates a moving-target problem. If agent capability improves quickly, benchmark design must move toward tasks with longer horizons, richer environments, and higher performance ceilings.

The lecture mentions recent hard benchmarks and notes that even benchmarks designed to be superhuman may have limited useful lifetimes. The underlying reason is that a fixed test set or finite task suite can eventually be memorized, saturated, or reduced to pattern matching.

Desired Properties of a Future-Proof Agent Environment

The lecture identifies five major properties:

Unbounded

An unbounded environment should not have a simple terminal state after which no further differentiation is possible. Factorio has a nominal win condition, launching a rocket, but performance can continue scaling far beyond it.

If $R(t)$ is production score at time t , then a saturated benchmark often has:

$$R(t) \leq R_{\max}$$

for a relatively small ceiling $R_{\max}$. A more useful open-ended benchmark has no practically reachable ceiling:

$$\forall R_0, \quad \exists \pi \text{ such that } R_{\pi}(t) > R_0$$

for sufficiently long horizon t , where π is an agent policy.

Procedural

A procedural environment reduces memorization. In Factorio, world maps are randomly generated. Resource patches, terrain, enemy bases, and spatial layouts vary. This prevents the agent from relying on a single memorized opening book.

The goal is to reduce the contribution of stored knowledge and increase the contribution of real-time problem solving. We can think of measured performance as:

$$\text{Measured Performance} = \text{Memorized Prior} + \text{In-Context Reasoning} + \text{Execution Skill} + \text{Error Recovery}$$

A procedural environment reduces the first term and stresses the remaining terms.

Long Horizon

The lecture emphasizes that frontier agents are moving toward one-hour tasks and beyond. Factorio trajectories may require thousands or tens of thousands of tool calls. This is qualitatively different from a benchmark where an agent emits one answer or writes one short program.

A long-horizon agent must manage:

- stale observations;
- accumulated side effects;
- growing namespaces;
- persistent bugs;
- increasingly abstract plans;
- compounding infrastructure investments.

Consequential

Actions must matter. In a consequential environment, a small mistake can impose future cost. Factorio has this property because factories are physical arrangements. A wrong placement can block future expansion, starve machines, break belts, or confuse the agent’s internal world model.

Open-Ended

The environment should permit many strategies rather than one fixed solution. Factorio supports:

- compact speedrun-style spaghetti factories;
- scalable bus-based factories;
- modular city-block factories;
- train-based logistics;
- solar-heavy low-pollution strategies;
- coal-heavy high-growth strategies requiring defense;
- human-like or non-human-like layout patterns.

Factorio as Physical Software Engineering

A recurring analogy is that Factorio resembles software engineering because it requires abstraction, modularity, interface discipline, and debugging. The difference is that the program is physical. Machines are functions, belts are dataflow edges, recipes are transformations, and resource patches are inputs.

A simple recipe can be modeled as a function:

$$f_{\text{furnace}}(\text{iron ore, fuel}) = \text{iron plate}$$

A larger factory is a directed graph:

$$G = (V, E)$$

where each node $v \in V$ is an entity such as a miner, furnace, assembler, inserter, belt, or chest, and each edge $e \in E$ represents item flow or dependency. A working factory is a graph that satisfies recipe constraints, spatial constraints, energy constraints, and throughput constraints.

The Factorio Learning Environment Harness

The contribution of the project is a harness that allows language-model agents and visual-language-model agents to interact with Factorio through code. The key design choice is that the agent does not use mouse and keyboard computer control. Instead, it uses a symbolic Python API.

The interaction loop is:

1. The model receives a system prompt containing the API schema and environment instructions.
2. The environment emits observations as a user message.
3. The model writes Python code as the assistant message.
4. The Python code calls high-level API functions.
5. Those calls are translated into low-level remote invocations.
6. Commands are sent over TCP using the RCON protocol to a Factorio headless server.
7. Factorio executes Lua scripts that mutate the game state.
8. The result is marshaled back into Python.
9. Standard output, standard error, observations, and reward updates are returned to the agent.

RCON and Hot-Loading Lua Scripts

The lecture explains that Factorio supports an administrative console mechanism through RCON, a TCP-based remote-control protocol used in games such as Minecraft. The FLE system uses this to send commands into a running Factorio headless multiplayer server.

The important engineering benefit is:

FLE does not require a custom Factorio server build. It can use the normal released headless server and replace the human client with an agent-facing Python client.

The system effectively hot-loads small Lua scripts into the running server. These scripts implement low-level game actions such as moving, placing, mining, picking up, and inspecting entities.

Symbolic and Visual Observations

Agents can observe the game in two main ways:

- **Symbolic observations:** structured Python objects, printed state, coordinates, entity lists, inventories, statuses, and errors.

- **Visual observations:** custom-rendered snapshots of the game produced by an in-house renderer.

The lecture reports that symbolic observations have generally been more useful than visual observations for most models. Models are good at reasoning over coordinates and typed objects. Images can confuse models, especially if the visual representation contains dynamic indicators, flashing lights, or irrelevant background detail.

However, the speaker suggests that newer multimodal models may now be strong enough to benefit from the visual affordance.

Name Space as Agent Memory

A crucial design detail is that agents operate in a persistent Python namespace. They can assign observations to variables, define functions, and reuse abstractions later. For example, an agent might observe a drill and chest, store them as variables, and later check whether they need fuel.

The namespace functions as a working memory:

$$\mathcal{M}_t = \{\text{variables, functions, classes, observed entities, plans}\}$$

At each step, the model can update this memory:

$$\mathcal{M}_{t+1} = \text{Update}(\mathcal{M}_t, o_t, a_t, e_t)$$

where o_t is the observation, a_t is the action, and e_t is any execution feedback or error.

Hierarchical Abstraction

The environment incentivizes agents to create reusable functions. This matters because an agent cannot scale to thousand-entity factories if every policy must explicitly manipulate every entity.

A good agent should compress repeated patterns into functions:

```
def automate_iron_plates(origin):
    ore = nearest(Resource.IronOre, origin)
    coal = nearest(Resource.Coal, origin)
    drill = place_entity(Entity.BurnerMiningDrill, ore.position)
    furnace = place_entity(Entity.StoneFurnace, offset(ore.position, dx=2, dy=0))
    insert_fuel(drill, coal)
    connect_output(drill, furnace)
    return {"drill": drill, "furnace": furnace}
```


This is not necessarily the exact lecture code, but it captures the policy style. The key idea is that a later action token can refer to a whole production subroutine instead of spelling out every primitive operation again.

The expressivity per generated token should increase over time:

$$\text{Effective Action Expressivity} = \frac{\text{Environment Work Performed}}{\text{Model Tokens Sampled}}$$

As agents build functions, this ratio can improve.

Hardware-Level Explanation

Execution Stack

The FLE stack can be understood as a multi-layer system:

1. **LLM inference layer:** samples policy code.
2. **Python execution layer:** runs the sampled code and exposes typed API objects.
3. **Tool extraction layer:** translates Python calls into primitive game operations.
4. **TCP/RCON communication layer:** sends commands to the game server.
5. **Lua execution layer:** runs scripts inside Factorio.
6. **Factorio simulation layer:** updates game state.
7. **Observation and reward layer:** returns symbolic objects, logs, errors, and production-score updates.

This architecture is analogous to a deep learning systems stack where a high-level program is lowered into primitive operations, executed by a runtime, and then measured.

CPU and GPU Requirements

The lecture states that FLE does not require a GPU for the Factorio simulation itself. The Factorio server runs on CPU, and an individual rollout can run on a single CPU core. The GPU is relevant only if the model inference is local and GPU-accelerated.

The performance bottleneck is dominated by LLM sampling, not Factorio simulation. If T_{sample} is model sampling latency and T_{env} is environment-step latency, the lecture suggests a regime roughly like:

$$T_{\text{sample}} \gg T_{\text{env}}$$

A rough ratio discussed is approximately:

$$\frac{T_{\text{sample}}}{T_{\text{env}}} \approx 9 \text{ to } 10$$

This means optimizing the server communication path has not been the highest priority so far. However, if distilled small models become fast enough, then environment throughput will matter more.

Simulation Throughput

The lecture reports that the system can run approximately 220 tool operations per second on average, with faster operations reaching around 600 operations per second. On an M4 laptop, the speaker suggests that one can run roughly 64 to 128 headless Factorio instances concurrently.

Let:

- N be the number of parallel Factorio servers;
- q be tool operations per second per server;
- C be the number of sampled model tokens per second;
- L be the average tokens per policy;
- k be the average primitive tool calls per policy.

Then the environment-side primitive capacity is:

$$\text{EnvOpsPerSecond} = Nq$$

The model-side policy generation rate is:

$$\text{PoliciesPerSecond} = \frac{C}{L}$$

The demanded tool-operation rate is:

$$\text{DemandedOpsPerSecond} = k \cdot \frac{C}{L}$$

The overall system is model-bound if:

$$k \cdot \frac{C}{L} < Nq$$

and environment-bound if:

$$k \cdot \frac{C}{L} > Nq$$

The current frontier-model setting is usually model-bound because sampling from large models is slow. A future distilled-agent setting may become environment-bound.

Turn-Based Versus Real-Time Regimes

The environment can be run in different timing modes:

- **Real-time mode:** the game continues running while the model samples. Slow reasoning has opportunity cost.
- **Paused or turn-based mode:** the game pauses during sampling, removing latency as a gameplay disadvantage.

Real-time mode creates an interesting trade-off:

$$\text{Agent Utility} = \text{Decision Quality} - \lambda \cdot \text{Decision Latency}$$

where λ measures how costly delay is in the current situation. A very intelligent but slow model can lose to a weaker but faster model if actions must be taken quickly, especially during attacks or other urgent events.

This is analogous to GPU systems where a theoretically optimal schedule may be useless if scheduling overhead dominates the actual kernel runtime.

Batching Rollouts

Because Factorio simulation is relatively cheap compared to LLM sampling, FLE naturally supports batched rollouts. Multiple servers can run in parallel, producing trajectories for evaluation, distillation, or reinforcement learning.

If M models are evaluated across S scenarios and R repeated seeds, the total number of rollouts is:

$$N_{\text{rollouts}} = M \cdot S \cdot R$$

For each rollout with H policy steps and k primitive tool calls per policy, the total primitive interactions are:

$$N_{\text{tool}} = M \cdot S \cdot R \cdot H \cdot k$$

This can quickly reach thousands or tens of thousands of tool calls per agent, which is the desired long-horizon evaluation regime.

Custom Renderer

Factorio's normal client is the renderer, but FLE avoids using the full client. The team built a custom renderer that uses Factorio sprite maps and a simplified tile system. The renderer is designed for agents rather than humans.

The custom renderer simplifies several things:

- removes unnecessary visual background detail;
- makes entity statuses more salient;
- avoids dynamic flashing indicators that confuse models;
- produces visual snapshots without a full client dependency;
- supports visual grounding for synthetic blueprints.

Visual Concept

Draw the FLE runtime as a pipeline: LLM policy generation on the left, Python API in the middle, TCP/RCON transport, Lua console, headless Factorio server, and finally symbolic plus visual observations flowing back to the model.

Mathematical Reasoning

Production Score as Reward

The lecture describes the reward as a production score derived from Factorio's internal item-value formula. The idea is to assign value to items based on the cheapest cost of producing their ingredients, plus modifiers for complexity and energy.

Let:

- $\mathcal{I}$ be the set of item types;
- v_i be the value of item i ;
- $P_i(t)$ be cumulative produced quantity of item i by time t ;
- $C_i(t)$ be cumulative consumed quantity of item i by time t .

Then a simplified production score is:

$$R(t) = \sum_{i \in \mathcal{I}} v_i (P_i(t) - C_i(t))$$

This resembles factory profit: producing valuable items increases score, while consuming items decreases score.

A generic recursive item-value formulation is:

$$v_i = \min_{r \in \mathcal{R}(i)} \left(\sum_{j \in \text{Inputs}(r)} a_{r,j} v_j + \alpha \cdot E_r + \beta \cdot K_r \right)$$

where:

- $\mathcal{R}(i)$ is the set of recipes that produce item i ;
- $a_{r,j}$ is the amount of ingredient j required by recipe r ;
- E_r is the energy or time cost of recipe r ;
- K_r is a complexity modifier;
- α and β weight energy and complexity.

The exact game formula is taken from Factorio's own internal code, but this abstraction captures the lecture's explanation.

Why Net Production Reduces Reward Hacking

If the reward were simply gross production,

$$R_{\text{gross}}(t) = \sum_{i \in \mathcal{I}} v_i P_i(t)$$

then an agent might exploit loops that create and destroy items. Net production penalizes consumed resources:

$$R_{\text{net}}(t) = \sum_{i \in \mathcal{I}} v_i (P_i(t) - C_i(t))$$

This makes it harder to gain score without building a genuinely productive system.

Throughput Task Objective

Lab-play throughput tasks require the agent to produce a target rate. For item i , let $Q_i([t_0, t_1])$ be the quantity produced during the measurement interval $[t_0, t_1]$. The measured throughput is:

$$\rho_i = \frac{Q_i([t_0, t_1])}{t_1 - t_0}$$

If the target is ρ_i^* , the success condition is:

$$\rho_i \geq \rho_i^*$$

For level-one item tasks, the lecture mentions targets such as 16 items per minute. For fluids, it mentions targets such as 250 units per minute.

The holdout period matters. If the agent only manually crafts items and briefly feeds a machine, it might hit the target for a moment. A holdout interval forces the production line to be somewhat sustainable:

$$\forall t \in [t_0, t_1], \quad \rho_i(t) \approx \rho_i^*$$

In practice, agents can still partially hack this by filling buffers, such as chests, before the holdout period.

Exponential Growth of Tech-Tree Costs

The lecture emphasizes that Factorio's progression gets harder because downstream items require increasingly many upstream resources. A simplified model is:

$$c_{\ell+1} \approx 2c_\ell$$

where c_ℓ is the resource cost of reaching level ℓ in the dependency or technology tree.

After L levels, the cost is approximately:

$$c_L \approx c_0 2^L$$

This exponential structure is one reason the environment resists saturation. Even if agents improve from producing basic circuits to advanced circuits, launching rockets and then scaling to mega-base throughput requires many additional doublings of competence.

Milestone Score

The lecture notes that pure production score can incentivize narrow behavior. For example, an agent might mine coal forever rather than progressing through the tech tree. Milestones are used to measure exploration and novel item creation.

A combined score could be written as:

$$J(t) = R(t) + \gamma M(t)$$

where:

- $R(t)$ is production score;
- $M(t)$ is milestone progress;

- γ controls the milestone weight.

Milestones help distinguish agents that merely exploit a basic resource loop from agents that create new capabilities.

Compounding Decision Quality

The lecture compares startup-like long-horizon success to making many slightly above-average decisions. A simple multiplicative model captures this intuition. Suppose each decision changes productivity by a factor $1 + \epsilon_t$. After T decisions:

$$P_T = P_0 \prod_{t=1}^T (1 + \epsilon_t)$$

If $\epsilon_t = \epsilon$ is small and positive:

$$P_T = P_0 (1 + \epsilon)^T$$

Using the approximation $\log(1 + \epsilon) \approx \epsilon$ for small ϵ :

$$P_T \approx P_0 e^{\epsilon T}$$

Thus small improvements in average decision quality produce large long-horizon differences. This explains why open-play production scores can separate models over thousands of steps.

Code Walkthrough

Minimal Policy: Place a Burner Mining Drill

The lecture shows a simple policy in which the agent finds coal, places a burner mining drill, observes entities, and performs an invariant check.

A representative version is:

```
coal = nearest(Resource.Coal)
move_to(coal.position)

drill = place_entity(
    Entity.BurnerMiningDrill,
    position=coal.position
)

entities = observe_entities(radius=10)
print("Nearby entities:", entities)
```

```
print("Placed drill:", drill)
```

```
assert drill.status == EntityState.Working
```

Line-by-line explanation:

- `nearest(Resource.Coal)` queries the symbolic world model for the nearest coal patch.
- `move_to(coal.position)` moves the embodied character near the resource because the character has limited reach.
- `place_entity(...)` invokes a primitive placement action through the Python API.
- The placement is lowered into a remote call and executed in Factorio through Lua.
- `observe_entities(radius=10)` returns nearby entities as structured objects.
- `print(...)` records state into standard output, which becomes part of the next observation stream.
- `assert drill.status == EntityState.Working` checks whether the agent's belief matches the environment.

This illustrates an important principle: strong agents should not simply act. They should act, observe, verify, and repair.

Defensive Policy with Error Handling

The lecture contrasts coding styles across models. Some models write fast exploratory scripts with many prints. Others write defensive code with try-except blocks and invariant checks.

A representative defensive policy is:

```
try:
    coal = nearest(Resource.Coal)
    if coal is None:
        raise RuntimeError("No coal found near current position.")

    move_to(coal.position)
    drill = place_entity(Entity.BurnerMiningDrill, coal.position)

    status = inspect_entity(drill).status
    print("Drill status after placement:", status)

    if status != EntityState.Working:
        print("Attempting repair: checking fuel and output blockage.")
```



```

        fuel_inventory = inspect_inventory(drill, Inventory.Fuel)
        print("Fuel inventory:", fuel_inventory)

except Exception as exc:
    print("Policy failed with exception:", repr(exc))
    print("Current position:", get_position())
    print("Inventory:", get_inventory())

```

The advantage is robustness. The disadvantage is token overhead. The policy may spend many tokens on defensive scaffolding rather than environment progress.

The trade-off can be modeled as:

$$\text{Expected Progress} = p_{\text{success}} \cdot W_{\text{action}} - C_{\text{tokens}} - C_{\text{latency}}$$

Defensive code can increase p_{success} but also increases token and latency cost.

Reusable Automation Function

A more scalable agent should create reusable production abstractions:

```

def build_burner_mining_pair(resource, output_offset=(1, 0)):
    patch = nearest(resource)
    move_to(patch.position)

    drill = place_entity(Entity.BurnerMiningDrill, patch.position)
    chest_pos = offset(patch.position, dx=output_offset[0], dy=output_offset[1])
    chest = place_entity(Entity.WoodenChest, chest_pos)

    rotate_entity(drill, direction_toward(chest.position))
    insert_fuel(drill, item=Item.Coal, count=5)

    observed = observe_entities(radius=8)
    print("Built mining pair:", drill, chest)
    print("Observed after build:", observed)

    return drill, chest

```

This function encodes a design pattern: miner plus output container. It can later be called for coal, iron ore, copper ore, or stone.

The hardware-systems analogy is a reusable kernel primitive. Just as CUDA programmers wrap a tiled load or warp-level reduction in a helper abstraction, the Factorio agent wraps a repeated build pattern in a function.

Monitoring and Refueling

Agents can use the namespace as memory and write utility functions to monitor factory health:

```
def refuel_burner_entities(entities, fuel_item=Item.Coal, target_count=10):
    for entity in entities:
        status = inspect_entity(entity).status
        if status in [EntityStatus.NoFuel, EntityStatus.LowFuel]:
            print("Refueling:", entity)
            move_to(entity.position)
            insert_fuel(entity, item=fuel_item, count=target_count)
```

This captures an important transition from direct construction to maintenance. Long-horizon agents must keep old infrastructure alive while building new infrastructure.

Throughput Measurement Sketch

A lab-play throughput task might measure whether the factory can produce enough of a target item:

```
def measure_throughput(item, seconds=30):
    start = get_production_count(item)
    sleep(seconds)
    end = get_production_count(item)
    per_minute = 60.0 * (end - start) / seconds
    print("Throughput for", item, "=", per_minute, "items per minute")
    return per_minute
```

```
throughput = measure_throughput(Item.IronPlate, seconds=30)
assert throughput >= 16.0
```

The holdout period is essential because it tests whether the production line runs over time instead of only producing a short burst.

Middle-Out Compression

The lecture mentions removing stale observations from long trajectories. A conceptual implementation is:

```
def compress_trajectory(messages, keep_prefix=4, keep_suffix=12):
    prefix = messages[:keep_prefix]
    suffix = messages[-keep_suffix:]

    summary = summarize_relevant_state(
        variables=current_namespace_summary(),
        milestones=current_milestones(),
        active_errors=recent_errors(),
```

```

        production=current_production_stats()
    )

    return prefix + [summary] + suffix

```

The reason is that old observations may no longer match the current game state. If the agent attends to stale information, it may act as if the world still contains resources, machines, or layouts that have since changed.

The failure mode can be written as belief-state drift:

$$\Delta_t = d(\hat{s}_t, s_t)$$

where $\hat{s}_t$ is the agent’s believed state and s_t is the true environment state. Stale observations increase Δ_t , which increases error probability.

Performance Intuition

Why Sampling Dominates Runtime

In current FLE evaluations, large frontier models dominate wall-clock time. The Factorio server is fast; Python and RCON overheads are secondary. This means that for large models, performance engineering should focus on:

- reducing reasoning-token count;
- encouraging concise policies;
- using reusable functions;
- compressing context;
- distilling competence into smaller models;
- batching rollouts across many servers.

This resembles the GPU-inference regime where the expensive component is model execution, not the environment.

Why Abstraction Improves Throughput

A low-level agent that writes every primitive action explicitly pays high token cost:

$$C_{\text{low}} = O(A)$$

where A is the number of primitive actions.

An agent that defines reusable abstractions pays:

$$C_{\text{abstract}} = O(B + K)$$

where B is the one-time cost of defining building blocks and K is the cost of calling them. If the same pattern is reused many times, abstraction wins.

For n repeated factory blocks, suppose direct construction costs c tokens per block. Direct cost is:

$$T_{\text{direct}} = nc$$

If a function costs d tokens to define and u tokens to call, abstracted cost is:

$$T_{\text{abstract}} = d + nu$$

Abstraction is beneficial when:

$$d + nu < nc$$

or:

$$n > \frac{d}{c - u}$$

This is the same amortization logic used in software libraries, code generation, and kernel template design.

Why Symbolic Observations Often Beat Images

Symbolic observations expose coordinates, types, statuses, and inventories directly. An LLM can reason over:

$$(x_1, y_1), (x_2, y_2), \Delta x, \Delta y$$

and decide where to place entities. Visual observations require the model to infer the same structure from pixels. Unless the vision model is very strong and well-aligned to Factorio’s visual distribution, symbolic state is more reliable.

A simple comparison:

Observation Type	Main Advantage
Symbolic	Direct coordinates, statuses, inventories, and object types.

Observation Type	Main Advantage
Visual	Spatial gestalt, layout pattern recognition, and blueprint grounding.

The lecture suggests that newer models may begin to extract meaningful value from images, especially when images are simplified by the custom renderer.

Failure Cascades

Long-horizon agents can fail through cascades. A single bad action creates an inconsistent state. The agent then observes errors. The presence of repeated errors in context may bias the model toward further bad actions.

A simplified recurrence is:

$$p_{t+1} = p_t + \eta e_t$$

where:

- p_t is probability of error at step t ;
- e_t indicates whether an error occurred;
- η measures how much error context increases future error probability.

This is not a formal theorem, but it captures the lecture's observation that some models, once confused, continue producing exceptions.

Open Play Versus Lab Play Failure Modes

In **open play**, agents may avoid local corruption by moving elsewhere and building a new subfactory. This can preserve production-score growth even if the world becomes messy.

In **lab play**, the constrained area leaves less room for recovery. A broken layout can permanently block progress. Thus lab play is more sensitive to precise spatial reasoning and error correction.

Model-Specific Coding Styles

The lecture describes model behavior differences:

- Some Claude-style agents behave like exploratory researchers: they print state frequently, write scripts quickly, and learn from outputs.
- GPT-style agents may write more defensive code, with try-except blocks and invariant checks.

- Gemini-style agents have shown willingness to create hierarchical abstractions and reusable utility methods.
- Weaker models may misunderstand the API, produce syntax errors, or perform irrelevant behavior such as placing many chests.

A key lesson is that coding competency strongly correlates with Factorio performance. This is expected because the agent’s action space is code.

Why Technology Investment Matters

In open play, agents that research better technologies can shift to higher-growth regimes. For example, electric mining drills increase production capacity. This creates a step change in future throughput.

Let $P(t)$ be production capacity. Research can be modeled as a discrete upgrade:

$$P(t^+) = \kappa P(t^-)$$

where $\kappa > 1$. After the upgrade, all future production benefits. Therefore, agents that sacrifice short-term score for research can dominate later.

This is similar to systems optimization where investing time in a better kernel, better compiler pass, or better data pipeline can produce long-run throughput gains.

Common Misconceptions

Misconception: Factorio Is Just a Game Benchmark

Factorio is a game, but FLE uses it as a systems-engineering environment. The agent is not evaluated on reflexes or mouse control. It is evaluated on symbolic reasoning, code generation, state management, automation design, and long-horizon planning.

Misconception: Launching a Rocket Solves the Benchmark

Launching a rocket is only the nominal win condition. The environment remains open-ended because factories can scale far beyond one rocket. Infinite technologies also create an unbounded progression structure.

Misconception: Visual Input Should Always Help

For many models, visual input does not help and may hurt. Symbolic state is often more useful because it exposes exact coordinates and statuses. Visual input becomes valuable only if the model can reliably parse the scene and connect it to action.

Misconception: More Context Is Always Better

Longer context can include stale observations. If the agent attends to old state, it may act on false beliefs. Compression and state summarization are not merely efficiency tricks; they improve correctness.

Misconception: Production Score Alone Is Enough

Production score can incentivize narrow behavior, such as mining one easy resource. Milestones and task design help measure exploration, progression, and capability acquisition.

Misconception: Reward Hacking Can Be Fully Eliminated

The lecture is clear that reward hacking can be reduced but not completely eliminated. Holdout periods make burst production less useful, but agents can still fill buffers and temporarily satisfy throughput tests.

Misconception: Self-Play or RL from Scratch Is the Obvious Solution

The speaker argues that starting from zero knowledge is inefficient. Existing large models already contain broad world knowledge and coding skill. FLE can use them to solve the cold-start problem and generate trajectories for later distillation or reinforcement learning.

Practical Takeaways

Takeaway 1: Long-Horizon Agent Evaluation Needs Stateful Environments

A strong agent benchmark should not be a static prompt set. It should have state, side effects, delayed reward, and persistent consequences. FLE is valuable because a bad action can damage the future.

Takeaway 2: Symbolic APIs Are Powerful for Embodied Agents

Instead of forcing agents to use pixels and mouse movements, FLE exposes a typed symbolic interface. This lets language models use their strengths: code generation, object reasoning, and structured debugging.

Takeaway 3: Reusable Abstractions Are Central

Agents must move from one-off scripts to reusable functions and classes. The ability to build and reuse abstractions is not cosmetic; it is necessary for scaling

to large factories.

Takeaway 4: Context Management Is Part of Intelligence

Agents must decide what to remember, what to print, what to summarize, and what to ignore. This is similar to engineering a long-running service: logs are useful, but stale logs can mislead.

Takeaway 5: Open-Ended Scores Separate Models Over Time

Short tasks may compress all models into similar success rates. Long open-play runs allow growth rates to separate. A slightly better agent compounds its advantage over many steps.

Takeaway 6: Distillation Is a Natural Next Step

Frontier models are slow and expensive. Once they can produce verified successful trajectories, those trajectories can be used to train smaller models specialized for Factorio. This may enable real-time agent behavior and online RL.

Takeaway 7: Factorio Is Useful for Alignment Research

FLE can operationalize hypotheses about instrumental convergence: resource acquisition, self-preservation, goal-content integrity, cognitive enhancement, and power-seeking behavior can be expressed inside the environment.

Project or Experiment Ideas

Experiment 1: Compare Symbolic and Visual Observations

Run the same lab-play tasks with symbolic-only, visual-only, and combined observations. Measure success rate, token usage, error rate, and production score.

Experiment 2: Measure the Value of Printing

Compare policies that print detailed state against policies that print minimal state. Track whether additional logs reduce future errors or merely increase context length.

Experiment 3: Function Reuse Ablation

Disable persistent function definitions and compare performance with agents allowed to define reusable functions. The hypothesis is that reusable abstractions improve long-horizon progress.

Experiment 4: Holdout Length Sensitivity

Evaluate throughput tasks with holdout periods of 30, 60, 120, and 300 seconds. Measure whether longer holdouts reduce buffer-based reward hacking.

Experiment 5: Real-Time Versus Turn-Based Evaluation

Run the same models in paused and unpaused modes. This measures whether smaller, faster models can outperform larger, slower models when decision latency matters.

Experiment 6: Error Cascade Analysis

Track the probability of future errors after an exception appears in context. Estimate whether models exhibit a consistency bias toward continuing failure patterns.

Experiment 7: Distill Successful Trajectories

Collect verified lab-play completions from frontier models. Fine-tune a smaller model on policy code, observations, and successful recovery behaviors. Then evaluate whether the smaller model generalizes to new seeds.

Experiment 8: Instrumental Convergence Tasks

Create tasks where the reward target is stored in a box the agent can manipulate. Test whether the agent preserves the intended goal or changes the environment to make reward easier to obtain.

Experiment 9: Blueprint Grounding

Generate synthetic factory blueprints, verify that they function, render them using the custom renderer, and train a model to connect visual patterns to build policies.

Experiment 10: Systems Throughput Benchmark

Profile the FLE stack itself. Measure latency contributions from model sampling, Python execution, RCON transport, Lua execution, simulation update, rendering, and observation serialization.

Connections to GPU and Deep Learning Systems

Rollout Throughput and GPU Inference

Although Factorio itself runs on CPU, large-scale FLE training or evaluation is a deep learning systems workload. The expensive stage is model inference. A

practical system must coordinate:

- many CPU simulation workers;
- one or more GPU inference servers;
- request batching;
- variable-length generation;
- streaming observations;
- trajectory storage;
- reward computation.

This resembles RLHF infrastructure and environment-parallel reinforcement learning.

Throughput Model

Let:

- B be inference batch size;
- r be generated tokens per second per GPU;
- L be average generated tokens per policy;
- N be number of environments;
- s be environment steps per second per environment.

The maximum policy throughput from inference is:

$$\Theta_{\text{model}} = \frac{r}{L}$$

The maximum environment throughput is:

$$\Theta_{\text{env}} = Ns$$

The system throughput is approximately:

$$\Theta = \min(\Theta_{\text{model}}, \Theta_{\text{env}})$$

Current FLE with frontier models is usually model-limited. Distilled small models could shift the bottleneck to simulation, communication, or rendering.

Analogy to Kernel Optimization

Factorio factory optimization resembles GPU kernel optimization in several ways:

GPU Systems Concept	Factorio Analogy
Memory bandwidth	Belt, inserter, pipe, and train throughput.
Arithmetic intensity	Value produced per unit of raw resource movement.
Kernel fusion	Combining production stages to reduce logistics overhead.
Tiling	Local factory blocks that reuse nearby resources.
Occupancy	Number of active machines doing useful work.
Load imbalance	Some production lines starve while others back up.
Synchronization	Inserters and belts coordinating item handoff.
Register pressure	Local inventory and limited character capacity.
Profiling	Inspecting stalled machines and production statistics.

The analogy is not exact, but it is pedagogically useful. Both domains involve throughput, bottlenecks, dependency graphs, resource movement, and compounding design choices.

Instrumental Convergence Research Direction

Why Factorio Is Relevant to Alignment

The lecture ends by proposing FLE as a platform for studying instrumental convergence. Instrumental convergence is the hypothesis that many agents, regardless of final goal, may develop similar intermediate drives because those drives help achieve many objectives.

The lecture mentions several such drives:

- **self-preservation;**
- **goal-content integrity;**
- **cognitive enhancement;**
- **resource acquisition;**
- **power seeking.**

Factorio naturally exposes measurable versions of these drives.

Resource Acquisition

Resource acquisition is directly measurable through inventory, production score, machine count, and factory throughput.

A resource-acquiring policy maximizes:

$$R_{\text{resources}}(t) = \sum_{i \in \mathcal{I}} w_i q_i(t)$$

where $q_i(t)$ is quantity of resource i and w_i is its value.

Cognitive Enhancement

In FLE, cognitive enhancement may appear as the agent creating better tools, abstractions, functions, maps, or monitoring systems. A proxy measure could be:

$$C(t) = \text{Number of reusable abstractions} + \text{Coverage of monitoring utilities} + \text{Reduction in tokens per successful}$$

An agent that writes better internal tooling can act more efficiently later.

Goal-Content Integrity

A possible experiment is to make the agent optimize the production of whatever item is placed in a box. If the agent can manipulate the box, it may replace a hard target with an easy target. This tests whether the agent preserves the intended objective or changes the objective channel.

Formally, suppose the intended target is g , but the measured target is read from environment state b :

$$g_{\text{measured}} = h(b)$$

If the agent can modify b , then it may optimize:

$$\max_{\pi} R(h(b))$$

by changing b rather than achieving the intended goal. This is a reward-specification failure.

Self-Preservation

In Factorio, self-preservation can be operationalized through avoiding biters, building defenses, preserving power, and avoiding states that terminate the episode. If the agent has an objective requiring future work, staying alive is instrumentally useful.

Distilled Model Organisms

The proposed research path is:

1. use strong frontier models to generate successful trajectories;
2. verify task completions automatically in lab play;
3. distill Factorio competence into smaller models;
4. connect the small models to online RL loops;
5. train narrow paperclip-maximizer-style agents;
6. test whether instrumental behaviors arise;
7. study whether these behaviors transfer to other domains;
8. use mechanistic interpretability or activation steering to reduce undesirable drives.

This is powerful because smaller models are cheaper to run, easier to analyze, and more practical for large-scale RL experiments.

Detailed Mental Models

Mental Model 1: Factorio as a Physical Program

A factory is a program written in belts, machines, inserters, and recipes. The input is raw resources. The output is high-value items. Bugs are physical misplacements. Refactoring is rebuilding layouts. Abstraction is a reusable construction function.

Mental Model 2: The Agent as a Long-Running Engineer

The agent is not answering one question. It is operating a long-running engineering project. It must inspect, build, debug, maintain, extend, and optimize.

Mental Model 3: Production Score as Economic Throughput

The reward is not arbitrary. It measures net value creation. A good factory behaves like a growing economy: it converts low-value raw inputs into high-value outputs while investing in future productive capacity.

Mental Model 4: Abstraction Is Compression of Action

A function is a compressed plan. The agent that writes useful functions increases the amount of environment work it can perform per generated token.

Mental Model 5: Long Horizons Reveal Growth Rates

In short tasks, models may look similar. In long tasks, small differences in planning, debugging, and abstraction compound. Open play separates models by their growth curves.

Final Mental Model

The Factorio Learning Environment turns agent evaluation into a long-horizon systems-engineering problem. An agent must use code to control an embodied character, build a physical production graph, manage persistent state, recover from errors, create reusable abstractions, and compound small engineering decisions into large throughput gains. The benchmark is difficult because progress is procedural, spatial, consequential, and open-ended. It is also valuable because it connects practical agent capability, code generation, long-context memory, online learning, reward hacking, and alignment-relevant instrumental behavior in one measurable environment.

A strong FLE agent is not merely a model that knows Factorio facts. It is a software engineer, factory planner, debugger, logistics optimizer, and long-horizon decision maker compressed into a policy that writes executable code under uncertainty.